

Paso de parámetros por referencia y repaso de POO

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Explicar qué recibe una función según el modo de paso: una copia, una dirección o un alias.
- Escribir funciones que modifican variables del llamador, con punteros en C y referencias en C++, y devolver más de un resultado por parámetros de salida.
- Reconstruir las piezas de una clase y pasar objetos a funciones por referencia.
- Deducir el rango de un tipo a partir de sus bits y compararlo con los lenguajes donde el número es un objeto.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA2: diseñar e implementar soluciones usando las herramientas del lenguaje.

Referencia bibliográfica

R. Thareja, *Data Structures Using C*, 2018, capítulo 6 (paso de argumentos usando punteros). R. Sebesta, *Concepts of Programming Languages*, 2015, capítulo 6 (tipos de datos primitivos), capítulo 9 (subprogramas y paso de parámetros) y capítulos 11 y 12 (encapsulación y orientación a objetos).

Contenido I

- 1 Qué viaja en una llamada
- 2 El paso por referencia
- 3 Repaso de POO
- 4 Los tipos por dentro
- 5 Ejercicios
- 6 Resumen y práctica

Contenido I

1 Qué viaja en una llamada

2 El paso por referencia

3 Repaso de POO

4 Los tipos por dentro

5 Ejercicios

6 Resumen y práctica

Lo que dejaron los punteros y los arreglos I

- El nombre del arreglo vale `&A[0]` y `A[i]` es `*(A + i)`: el corchete es aritmética de direcciones.
- En la llamada `suma(A, 5)` viajó una dirección, no una copia del arreglo; por eso la función necesitó que le dijeran `n`.
- Una matriz vive fila tras fila, y la del tamaño justo se arma con un solo bloque de `malloc`.
- Pregunta pendiente: si el arreglo viaja como dirección, ¿qué viaja cuando el argumento es un `int` suelto?

Hoy la pregunta es qué viaja exactamente en cada llamada. La respuesta explica un misterio viejo del `scanf` y ordena, al final, el trabajo con objetos.

¿Qué imprime? I

Dos variables quieren cambiar de lugar:

```
void intercambiar(int a, int b) {  
    int temporal = a;  
  
    a = b;  
    b = temporal;  
}  
  
int main(void) {  
    int x = 10;  
    int y = 2;  
  
    intercambiar(x, y);  
    printf("x = %d y = %d\n", x, y);  
    return 0;  
}
```

En parejas: ¿qué imprime?

¿Qué imprime?: respuesta I

```
x = 10 y = 2
```

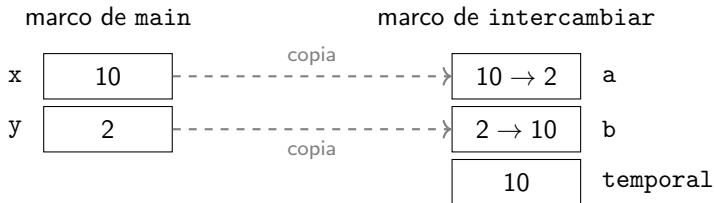
La función `corrió`, `temporal` hizo su baile y las variables quedaron igual.

- Dentro de la función, `a` y `b` sí terminaron intercambiadas.
- El problema no es el intercambio: es *quiénes son* `a` y `b`.

Una copia viaja I

Paso por valor (Sebesta, cap. 9)

En el paso por valor, la llamada evalúa cada argumento y copia el resultado en una variable local nueva de la función. La función trabaja con sus copias, y las copias mueren cuando la función retorna.



- a nace como copia de x; desde ahí, cada una vive en su celda.

Una copia viaja II

- El intercambio ocurrió en el marco de `intercambiar`, y ese marco entero se destruye al retornar: el trabajo se fue con él.
- La única puerta de salida de una función por valor es su `return`.

¿Qué imprime? I

Ahora con un puntero en escena. Primero la función:

```
int algo(int a, int b) {  
    int ans = a + b;  
  
    a = a + 1;  
    b = b - 1;  
    return ans;  
}
```

¿Qué imprime? II

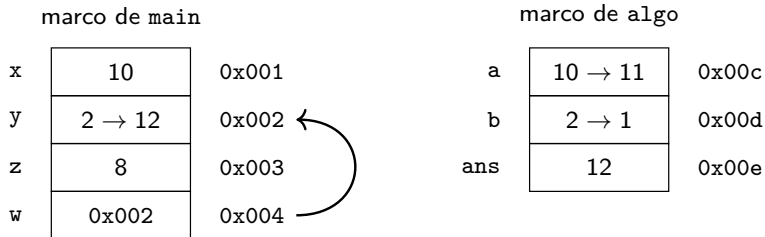
Y el main que la llama:

```
int main(void) {  
    int x = 10;  
    int y = 2;  
    int z = 8;  
    int *w = &y;  
  
    printf("x = %d y = %d z = %d\n", x, y, z);  
    *w = algo(x, y);  
    printf("x = %d y = %d z = %d\n", x, y, z);  
    return 0;  
}
```

En parejas: dibuje los dos marcos con sus celdas y prediga las dos líneas.

¿Qué imprime?: respuesta I

```
x = 10 y = 2 z = 8  
x = 10 y = 12 z = 8
```



- x no cambió: a era una copia y el $a = a + 1$ se quedó en el marco de algo.
- y sí cambió, pero no lo hizo algo: lo hizo main al escribir $*w = 12$ con el valor retornado. El paso fue por valor de principio a fin y el puntero w es asunto del llamador.

Contenido I

1 Qué viaja en una llamada

2 El paso por referencia

3 Repaso de POO

4 Los tipos por dentro

5 Ejercicios

6 Resumen y práctica

La dirección viaja I

Para que la función alcance *x*, no se le manda el valor: se le manda *dónde vive*.

Paso por referencia con punteros (Thareja, cap. 6)

El argumento es la dirección de la variable y el parámetro es un puntero. La función recibe una copia de la dirección, y con ella lee y escribe la celda original del llamador.

```
void intercambiar(int *a, int *b) {  
    int temporal = *a;  
  
    *a = *b;  
    *b = temporal;  
}
```

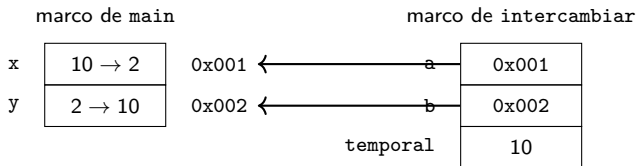
La dirección viaja II

La llamada entrega las direcciones con &:

```
int main(void) {  
    int x = 10;  
    int y = 2;  
  
    intercambiar(&x, &y);  
    printf("x = %d y = %d\n", x, y);  
    return 0;  
}
```

```
x = 2 y = 10
```


La dirección viaja: el dibujo I



- Lo copiado sigue siendo el argumento; solo que ahora el argumento es una dirección, y una copia de la dirección llega a la misma casa.
- Las dos marcas del modo: & en la llamada (entregar la dirección) y * adentro (usar la celda apuntada).
- *a = *b escribe en 0x001: la x de main cambia aunque main no haya movido un dedo.

¿Qué imprime? I

El mismo programa de la traza anterior, con un solo cambio en la firma:

```
int algo(int *a, int b) {  
    int ans = *a + b;  
  
    *a = *a + 1;  
    b = b - 1;  
    return ans;  
}
```

¿Qué imprime? II

Y la llamada ahora entrega la dirección de x:

```
int main(void) {  
    int x = 10;  
    int y = 2;  
    int z = 8;  
    int *w = &y;
```

```
    printf("x = %d y = %d z = %d\n", x, y, z);  
    *w = algo(&x, y);  
    printf("x = %d y = %d z = %d\n", x, y, z);  
    return 0;  
}
```

```
int algo(int *a, int b) {  
    int ans = *a + b;  
  
    *a = *a + 1;  
    b = b - 1;  
    return ans;  
}
```

En parejas: ¿qué imprime ahora?

¿Qué imprime?: respuesta I

```
x = 10 y = 2 z = 8  
x = 11 y = 12 z = 8
```

Quién cambió a quién:

Variable	¿Cambió?	Quién escribió
x	10 → 11	algo, vía *a
y	2 → 12	main, vía *w
z	no	nadie

- a guarda 0x001: el `*a = *a + 1` viaja hasta la celda de x.
- b sigue siendo copia: el `b = b - 1` muere con el marco.
- Una misma firma puede mezclar los dos modos: a por referencia, b por valor.

El misterio del & en scanf I

Desde la primera lectura de datos se escribe

```
scanf("%d", &n);
```

y el & iba por fe. Ya no:

- `scanf` necesita *escribir* lo leído en `n`: es una función que modifica variables del llamador.
- Por eso recibe la dirección `&n`: paso por referencia con punteros, el mismo de intercambiar.
- `printf` solo necesita leer el valor, y por eso `n` viaja sin `&`: paso por valor.

El return se queda corto !

Una función devuelve *un* valor. ¿Y si el recorrido necesita entregar dos, como el menor y el mayor de un arreglo en una sola pasada?

```
void minimo_y_maximo(int *A, int n,  
                    int *minimo, int *maximo) {  
    int i = 1;  
  
    *minimo = A[0];  
    *maximo = A[0];  
    while (i < n) {  
        if (A[i] < *minimo) {  
            *minimo = A[i];  
        }  
        if (A[i] > *maximo) {  
            *maximo = A[i];  
        }  
        i = i + 1;  
    }  
}
```

El return se queda corto II

El llamador presta dos celdas para las respuestas:

```
int main(void) {  
    int A[6] = {7, 3, 9, 1, 8, 4};  
    int menor, mayor;  
  
    minimo_y_maximo(A, 6, &menor, &mayor);  
    printf("minimo = %d maximo = %d\n", menor, mayor);  
    return 0;  
}
```

```
minimo = 1 maximo = 9
```

- A y n son entradas; minimo y maximo son *parámetros de salida*: direcciones donde la función deja sus resultados.
- El llamador declara menor y mayor y presta sus celdas con &.
- Una pasada, dos respuestas, y el return quedó libre (aquí no hizo falta).

Sin & y sin novedad I

Una función que *escribe* un arreglo del llamador:

```
/* Escribe en A los primeros n numeros pares: 0, 2, 4, ... */  
void llenar_pares(int *A, int n) {  
    int i = 0;  
  
    while (i < n) {  
        A[i] = 2 * i;  
        i = i + 1;  
    }  
}
```


Sin & y sin novedad II

El llamador entrega el arreglo por su nombre:

```
int main(void) {  
    int A[5];  
    int i;  
  
    llenar_pares(A, 5);  
    i = 0;  
    while (i < 5) {  
        printf("%d ", A[i]);  
        i = i + 1;  
    }  
    printf("\n");  
    return 0;  
}
```

0 2 4 6 8

Sin & y sin novedad III

- En la llamada no hay &: el nombre A ya vale &A[0]. Los arreglos nacen con la dirección puesta.
- Por eso toda función sobre arreglos ha podido leerlos y modificarlos desde el principio: siempre viajaron por referencia.
- El `int` suelto es el que necesita el &; el arreglo, no.

La matriz viaja igual I

La matriz del tamaño justo es un bloque único, y un bloque viaja como cualquier arreglo: su dirección y sus dos tamaños.

```
void llenar(int *M, int n, int m);
```

- Adentro se sigue escribiendo $M[i * m + j]$: la cuenta del orden por filas no cambia por estar en una función.
- La función escribe directo sobre el bloque del llamador: nada se copia, sin importar si la matriz tiene doce celdas o un millón.

¿Qué imprime? I

C++ agrega una tercera vía. Primero, la pieza nueva:

```
int main() {  
    int x = 0;  
    int y = 10;  
    int &z = x;  
  
    printf("x = %d y = %d z = %d\n", x, y, z); 0, 10, 0  
    z = y + x;  
    printf("x = %d y = %d z = %d\n", x, y, z); 10, 10, 10  
    return 0;  
}
```

En parejas: ¿qué imprime? La pista está en que z no es una variable nueva.

¿Qué imprime?: respuesta I

```
x = 0 y = 10 z = 0  
x = 10 y = 10 z = 10
```

x	z	0 → 10	0x001
	y	10	0x002

- `int &z = x` no crea una celda: le pone un segundo nombre a la celda de `x`. `z` es un *alias*.
- `z = y + x` escribe 10 en 0x001, y por eso `x` amanece en 10.
- La referencia se ata una vez, al nacer, y no se puede redirigir después: no es un puntero que se mueve, es un nombre fijo.

El intercambio en C++ I

El alias entra a la firma y el intercambio queda limpio:

```
void intercambiar(int &a, int &b) {  
    int temporal = a;  
  
    a = b;  
    b = temporal;  
}
```

El intercambio en C++ II

Y la llamada va sin marcas:

```
int main() {  
    int x = 10;  
    int y = 2;  
  
    intercambiar(x, y);  
    printf("x = %d y = %d\n", x, y);  
    return 0;  
}
```

x = 2 y = 10

- Durante la llamada, a es x: otro nombre para la misma celda.
- El cuerpo se lee igual que la versión por valor, sin asteriscos; la llamada se lee igual que la que falló.

El intercambio en C++ III

La llamada no avisa

En C, `intercambiar(&x, &y)` grita que `x` puede cambiar. En C++, `intercambiar(x, y)` puede ser por valor o por referencia: para saber si el argumento corre peligro hay que leer la *firma* de la función.

Tres maneras de pasar un `int` I

	Por valor	Por puntero	Por referencia
La función recibe	una copia	una dirección	un alias
La llamada	<code>f(x)</code>	<code>f(&x)</code>	<code>f(x)</code>
En la firma	<code>int a</code>	<code>int *a</code>	<code>int &a</code>
Adentro se usa	<code>a</code>	<code>*a</code>	<code>a</code>
¿Puede cambiar <code>x</code> ?	no	sí	sí
Disponible en	C y C++	C y C++	solo C++

- Los tres modos copian *algo*: el valor, la dirección o nada visible (el alias se resuelve con una dirección por debajo).
- En el curso: C usa valor y punteros; C++ suma las referencias, que serán la forma natural de mover objetos.

Contenido I

- 1 Qué viaja en una llamada
- 2 El paso por referencia
- 3 Repaso de POO**
- 4 Los tipos por dentro
- 5 Ejercicios
- 6 Resumen y práctica

Del paquete de datos al objeto I

Un estudiante del curso son un código y sus notas. Con lo visto hasta aquí, esos datos andarían sueltos:

```
int codigo;  
double notas[3];  
int registradas;  
  
registrar_nota(notas, &registradas, 4.0);
```

Tres variables que deben moverse juntas, y una función aparte que cualquiera puede saltarse escribiendo `notas[0]` directo.

La clase y el objeto (Sebesta, caps. 11–12)

Una clase reúne en una sola pieza los datos de una entidad (*atributos*) y las operaciones que los manejan (*métodos*). Un objeto es un ejemplar concreto de la clase, con sus propios valores en los atributos.

La clase Estudiante I

Los atributos, el constructor y la operación que escribe:

```
class Estudiante {  
private:  
    int codigo;  
    double notas[3];  
    int registradas;  
  
public:  
    Estudiante(int c) {  
        codigo = c;  
        registradas = 0;  
    }  
  
    void registrar_nota(double nota) {  
        if (registradas < 3) {  
            notas[registradas] = nota;  
            registradas = registradas + 1;  
        }  
    }  
}
```

La clase Estudiante II

Y las operaciones que consultan:

```
double promedio() {  
    double suma = 0;  
    int i = 0;  
  
    while (i < registradas) {  
        suma = suma + notas[i];  
        i = i + 1;  
    }  
    return suma / registradas;  
}  
  
int obtener_codigo() {  
    return codigo;  
}  
};
```

- Los atributos van en private: solo los métodos de la clase pueden tocarlos.

La clase Estudiante III

- El *constructor* `Estudiante(int c)` corre al crear el objeto y lo deja en un estado sano: cero notas registradas.
- `registrar_nota` valida antes de escribir: el arreglo no se desborda porque la clase no lo permite.

El objeto en acción I

```
int main() {  
    Estudiante e(1023);  
  
    e.registrar_nota(4.0);  
    e.registrar_nota(3.5);  
    e.registrar_nota(4.5);  
    printf("Estudiante %d: promedio %.2f\n",  
          e.obtener_codigo(), e.promedio());  
    return 0;  
}
```

Estudiante 1023: promedio 4.00

- Estudiante e(1023) reserva las celdas del objeto y ejecuta el constructor.
- e.registrar_nota(4.0) es una llamada con un participante implícito: el objeto e, cuyos atributos el método lee y escribe.

¿Y si main toca las notas? I

En parejas: ¿compila?

```
int main() {  
    Estudiante e(1023);  
  
    e.notas[0] = 5.0;  
    return 0;  
}
```


¿Y si main toca las notas?: respuesta I

No compila. El compilador responde:

```
error: 'double Estudiante::notas [3]' is private  
    within this context  
19 |         e.notas[0] = 5.0;
```

- `private` cierra la puerta: las notas solo cambian pasando por `registrar_nota`, que valida el cupo.
- Esto es la *encapsulación*: los datos quedan al cuidado de sus propios métodos, y el compilador es el portero.
- Así como la condición del ciclo cuidaba los límites del arreglo, la clase cuida los suyos, y esta vez el error se atrapa *antes* de ejecutar.

¿Qué imprime? I

El objeto también viaja a las funciones, y el modo de paso vuelve a decidirlo todo:

```
/* Recibe una copia del estudiante */
void premiar_copia(Estudiante e) {
    e.registrar_nota(5.0); ← [3, 4, 8]
}

/* Recibe el estudiante mismo */
void premiar(Estudiante &e) {
    e.registrar_nota(5.0);
}
```

¿Qué imprime? II

Y el main que las llama:

```
int main() {  
    Estudiante e(1023);  
  
    e.registrar_nota(4.0);  
    e.registrar_nota(3.0);  
    printf("promedio: %.2f\n", e.promedio());  
    premiar_copia(e);  
    printf("tras premiar_copia: %.2f\n", e.promedio());  
    premiar(e);  
    printf("tras premiar: %.2f\n", e.promedio());  
    return 0;  
}
```

Handwritten annotations in red:

- Arrow from `e(1023)` to `[4, 3, -]` with `arg=2` next to it.
- Red circle around `e.promedio()` in the first `printf` call, with `3.5` written above it.
- Red circle around `e.promedio()` in the second `printf` call, with `3.5` written above it.
- Red circle around `e.promedio()` in the third `printf` call, with `4.0` written below it.

En parejas: las tres líneas.

¿Qué imprime?: respuesta I

```
promedio: 3.50  
tras premiar_copia: 3.50  
tras premiar: 4.00
```

- `premiar_copia` copió el objeto entero —código, notas y contador— y la nota 5.0 quedó en la copia, que murió con el marco.
- `premiar` recibió un alias: la nota quedó en el original y el promedio subió a $\frac{4,0+3,0+5,0}{3} = 4,0$.
- Copiar un objeto copia *todos* sus atributos; la referencia cuesta lo que una dirección. Por eso los objetos viajan por referencia.

El promedio de nadie I

Un objeto recién creado todavía no tiene notas. ¿Qué responde `promedio()`?

```
Estudiante e(1023);  
  
printf("promedio sin notas: %.2f\n", e.promedio());
```

```
promedio sin notas: -nan
```

La operación existe, la respuesta no

Con registradas en 0, la división $0/0$ no es un número y el programa imprime basura sin fallar. La clase protege *cómo* se guardan los datos, pero aún no dice *qué* se puede pedir y en qué estado. Ese contrato —operaciones, condiciones y significado— es el tema de la abstracción de datos.

Relaciones

Herencia: Una clase toma atributos y metodos de otra clase, agregar mas o sobrecargar. Simple: hereda de una sola clase (Java). Multiple: Puedo heredar de multiples.

Uso: Una clase usa metodos de otra.

Composición: Una clase esta compuesta por otras.

Polimorfimos: En un mecanismo de herencia un metodo puede diferentes comportamientos.

Mamifero ma1 = new Gato() ; Mamifero ma2 = new Perro();

Contenido I

- 1 Qué viaja en una llamada
- 2 El paso por referencia
- 3 Repaso de POO
- 4 Los tipos por dentro**
- 5 Ejercicios
- 6 Resumen y práctica

¿Qué se copió exactamente? I

Cuando `intercambiar(x, y)` recibió los enteros por valor, se copió algo. ¿Cuánto pesa ese algo, y qué cabe adentro?

```
int a = 5;
```

- La celda mide 4 bytes, que son 32 bits.
- Con 32 bits se escriben $2^{32} = 4\,294\,967\,296$ combinaciones de ceros y unos, ni una más.
- Esas combinaciones son todos los valores que el tipo podrá tomar en su vida.

Falta decir cuáles: ¿de 0 en adelante, o repartidos entre negativos y positivos?

El rango sale de los bits I

Rango de un entero de b bits

Sin signo, los valores van de 0 a $2^b - 1$. Con signo, en complemento a dos, van de -2^{b-1} a $2^{b-1} - 1$.

Tipo	Bits	Rango
char	8	$[-128, 127]$
unsigned char	8	$[0, 255]$
int	32	$[-2\,147\,483\,648, 2\,147\,483\,647]$
unsigned int	32	$[0, 4\,294\,967\,295]$
long	64	$[-2^{63}, 2^{63} - 1]$

- El signo se paga con la mitad del rango, y hay un negativo más que positivos porque el cero se cuenta del lado de arriba.
- Nada de esto lo decide el lenguaje: es cuántos bits hay.

La máquina lo confirma I

Los topes no hay que memorizarlos. Vienen escritos en `limits.h`:

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("int  %d bytes  [%d, %d]\n",
           (int) sizeof(int), INT_MIN, INT_MAX);
    printf("long %d bytes  [%ld, %ld]\n",
           (int) sizeof(long), LONG_MIN, LONG_MAX);
    return 0;
}
```

```
int  4 bytes  [-2147483648, 2147483647]
long 8 bytes  [-9223372036854775808, 9223372036854775807]
```

El mismo `sizeof` que decidía cuánto avanza `p + 1` es el que decide hasta dónde llega el tipo.

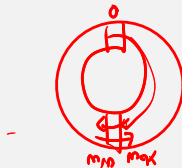
¿Qué imprime? I

En parejas, antes de correrlo:

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    int mayor = INT_MAX;
    unsigned int tope = UINT_MAX;
    int paso = 1;

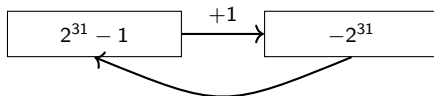
    printf("INT_MAX + 1    = %d\n", mayor + paso);
    printf("UINT_MAX + 1   = %u\n", tope + paso);
    return 0;
}
```



La pista está en que después del último valor no hay ninguno.

¿Qué imprime?: respuesta I

```
INT_MAX      = 2147483647
INT_MAX + 1  = -2147483648
UINT_MAX     = 4294967295
UINT_MAX + 1 = 0
```



El contador da la vuelta y no avisa

Sumarle uno al mayor `int` aterriza en el menor, como el odómetro de un carro que vuelve a ceros. Ni el compilador ni el programa dicen nada, igual que cuando se lee `A[4]` en un arreglo de cuatro: el resultado es un número perfectamente válido y perfectamente equivocado.

¿Qué imprime?: respuesta II

En `unsigned` la vuelta está en la definición del tipo; en `int` es lo que hace la máquina, no algo que el lenguaje prometa. El único cuidado real es el del programador: si la suma puede pasarse, el tipo se elige más ancho.

Los reales tampoco caben I

Un float son 4 bytes y un double son 8, repartidos entre el signo, el exponente y los dígitos. Con esos bits contados hay que representar una recta infinita.

```
double a = 0.1;
double b = 0.2;

printf("0.1 + 0.2 (double) = %.20f\n", a + b);
printf("0.3          (double) = %.20f\n", 0.3);
```

```
0.1 + 0.2 (double) = 0.30000000000000004441
0.3          (double) = 0.29999999999999998890
La comparacion dice que son distintos
0.1 + 0.2 (float)  = 0.30000001192092895508
```

Los reales tampoco caben II

- En base 2, $0,1$ es periódico, como $1/3$ en base 10: escribirlo exacto pide infinitos dígitos y solo hay 53.
- Lo que se guarda es el vecino más cercano, y los errores de cada operando se acumulan.
- El `float` falla antes: con la mitad de los bits, la basura aparece en el séptimo decimal en vez del decimoséptimo.

Los reales no se comparan con `==`

`a + b == 0.3` es falso y el programa tiene razón: son dos números distintos. Lo que se pregunta es si están cerca, $|a - b| < \varepsilon$ con un ε del tamaño del problema.

Cuando el número es un objeto I

El mismo desbordamiento, en Python:

```
import sys

print(2 ** 100)
print(sys.getsizeof(0), sys.getsizeof(1000), sys.getsizeof(2
↪ ** 100))
print(0.1 + 0.2)
```

```
1267650600228229401496703205376
28 28 40
0.30000000000000004
```


Cuando el número es un objeto II

- No hay desbordamiento porque no hay 4 bytes: un entero de Python es un *objeto*, con su encabezado y un arreglo de dígitos que crece pidiendo más memoria.
- El precio son 28 bytes para guardar un cero, siete veces lo que cuesta un `int`, y una suma que ya no es una instrucción del procesador.
- El real no se salvó: es el mismo `double` de C, con el mismo `0.30000000000000004`. Lo que cambió fue el entero.

La lista tampoco es un bloque de números

`sys.getsizeof([1, 2, 3, 4, 5])` responde 104, y los cinco números no están ahí: la lista guarda cinco direcciones de 8 bytes y cada entero vive aparte, con sus 28.

Java tiene las dos cosas I

`int` es un primitivo de 4 bytes; `Integer` es una clase, y su instancia es un objeto con dirección propia.

```
int mayor = Integer.MAX_VALUE;
System.out.println(mayor + 1);
System.out.println(Double.MIN_VALUE);

Integer a = Integer.valueOf("127");
Integer b = Integer.valueOf("127");
Integer c = Integer.valueOf("128");
Integer d = Integer.valueOf("128");
System.out.println((a == b) + " " + (c == d) + " " +
    ↪ c.equals(d));
```

```
-2147483648
4.9E-324
true false true
```

Java tiene las dos cosas II

- El primitivo desborda igual que en C: mismos 32 bits, misma vuelta al menor.
- `Double.MIN_VALUE` no es el más negativo, es el positivo más pequeño que se puede representar. El más negativo es `-Double.MAX_VALUE`.

== entre objetos compara direcciones

Con 127 da true y con 128 da false, y los valores no tienen nada que ver: Java guarda en caché los Integer de -128 a 127 , así que los dos primeros son el mismo objeto y los otros dos son objetos distintos con el mismo contenido. Para comparar valores está `equals`.

Lo que cuesta la comodidad I

	int A[5] en C	[1, 2, 3, 4, 5] en Python
Lo que ocupa	20 bytes seguidos	104 bytes de direcciones
Cada elemento	4 bytes del bloque	un objeto aparte, 28 bytes
A + 1 avanza	4 bytes	no hay paso fijo
Fuera de rango	silencio	IndexError
Al pasarse del tope	da la vuelta	no hay tope

- Lo que se gana —sin desbordamiento, sin `sizeof`, sin liberar— se paga en memoria y en saltos: los objetos están dispersos y cada acceso puede costar una traída de caché.
- Que indexar cueste $O(1)$ depende de que las casillas sean del mismo tamaño y estén pegadas. Sobre un arreglo de direcciones la cuenta sigue sirviendo para llegar a la dirección, pero el dato está en otra parte.

Las estructuras que vienen se construyen sobre el bloque de bytes, que es donde esa cuenta se puede hacer.

Contenido I

- 1 Qué viaja en una llamada
- 2 El paso por referencia
- 3 Repaso de POO
- 4 Los tipos por dentro
- 5 Ejercicios**
- 6 Resumen y práctica

Ejercicio I

En parejas: escriba la función

```
void dividir(int a, int b, int *cociente, int *residuo);
```

que deje en las salidas el cociente y el residuo de a entre b, de modo que este main imprima $17 = 3 * 5 + 2$:

```
int main(void) {  
    int c, r;  
  
    dividir(17, 5, &c, &r);  
    printf("17 = %d * 5 + %d\n", c, r);  
    return 0;  
}
```

Antes de escribir: ¿cuáles parámetros son de entrada y cuáles de salida?

Ejercicio: respuesta I

```
/* Deja en *cociente y *residuo el resultado  
de dividir a entre b */  
void dividir(int a, int b, int *cociente, int *residuo) {  
    *cociente = a / b;  
    *residuo = a % b;  
}
```

$$17 = 3 * 5 + 2$$

- a y b entran por valor: solo se leen.
- cociente y residuo son salidas por dirección, la misma receta de minimo_y_maximo.
- Dos resultados de una división que la máquina calcula junta: una firma así los entrega en una sola llamada.

Ejercicio I

En parejas: agregue a la clase Estudiante el método

```
double mejor_nota();
```

que devuelve la más alta de las notas registradas.

¿Qué recorrido conocido es este, y sobre qué arreglo corre?

Ejercicio: respuesta I

```
double mejor_nota() {  
    double mejor = notas[0];  
    int i = 1;  
  
    while (i < registradas) {  
        if (notas[i] > mejor) {  
            mejor = notas[i];  
        }  
        i = i + 1;  
    }  
    return mejor;  
}
```

mejor nota: 4.5

- El recorrido de buscar el mayor, ahora dentro de la clase: corre sobre los atributos propios y no recibe parámetros.
- Con cero notas, notas[0] es basura: el contrato pendiente crece.

Ejercicio de cierre I

Escriba un programa completo que:

- 1 Lea n y reserve en el montículo un arreglo de n reales (double).
- 2 Con void llenar(double *A, int n), lea los n valores usando scanf.
- 3 Con void estadisticas(double *A, int n, double *promedio, double *maxima), calcule el promedio y la nota más alta en una sola pasada.
- 4 Imprima los dos resultados y devuelva la memoria.

Con las verificaciones de siempre: el NULL de malloc, y tras el free, el puntero a NULL.

Contenido I

- 1 Qué viaja en una llamada
- 2 El paso por referencia
- 3 Repaso de POO
- 4 Los tipos por dentro
- 5 Ejercicios
- 6 Resumen y práctica

Qué viaja en cada llamada

- Por valor viaja una copia: la función no alcanza las variables del llamador y su única salida es el `return`.
- Por puntero viaja la dirección: `&` en la llamada, `*` adentro, y la función escribe las celdas originales; así trabaja `scanf` y así se devuelven varios resultados.
- Los arreglos siempre viajaron por referencia: el nombre ya es la dirección.
- La referencia de C++ es un alias fijo: firma con `&`, llamada limpia, y la firma es la que dice si el argumento puede cambiar.

Objetos y tipos

- Una clase junta atributos y métodos; `private` deja los datos al cuidado de sus métodos, y los objetos viajan por referencia.
- El rango de un tipo sale de sus bits, pasarse de él da la vuelta en silencio y los reales no se comparan con `==`; donde el número es un objeto no hay tope, y se paga en memoria y en saltos.

Lo que sigue: la abstracción de datos —separar lo que un tipo ofrece de cómo está construido— y los tipos abstractos de datos.

Referencias I

-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 6 (paso de argumentos usando punteros).
-  R. Sebesta. *Concepts of Programming Languages*. Pearson, 2015. Capítulo 6 (tipos de datos primitivos); capítulo 9 (subprogramas y paso de parámetros); capítulos 11 y 12 (encapsulación y orientación a objetos).
-  N. Kalicharan. *Data Structures in C*. 2008. Funciones y punteros.

Propuesto 1 I

Sin correrlo: ¿qué imprime? Es la traza de la sesión, con la firma en su tercera variante.

```
int algo(int &a, int b) {  
    int ans = a + b;  
  
    a = a + 1;  
    b = b - 1;  
    return ans;  
}
```

Propuesto 1 II

```
int main() {  
    int x = 10;  
    int y = 2;  
    int z = 8;  
    int *w = &y;  
  
    printf("x = %d y = %d z = %d\n", x, y, z);  
    *w = algo(x, y);  
    printf("x = %d y = %d z = %d\n", x, y, z);  
    return 0;  
}
```

Dibuje los marcos antes de responder.

Propuesto 2 I

Escriba la función

```
void intercambiar_extremos(int *A, int n);
```

que intercambia la primera y la última casilla de A reutilizando la función `intercambiar` de la sesión, sin mover datos por su cuenta. Pruébela con {1, 2, 3, 4, 5}.

Propuesto 3 I

Escriba la clase Rectangulo con base y altura privadas, constructor, y los métodos `area()`, `perimetro()` y `escalar(int factor)`, que multiplica ambos lados por el factor.

Con base 3 y altura 4, escale por 2. Antes de correr: ¿el área se duplica? ¿Y el perímetro?

Propuesto 4 I

Sin correrlo: ¿qué imprime?

```
void misterio(int a, int *b) {  
    a = a + *b;  
    *b = *b + 1;  
}  
  
int main(void) {  
    int u = 3;  
    int v = 5;  
  
    misterio(u, &v);  
    printf("%d %d\n", u, v);  
    return 0;  
}
```

¿Cuál de los dos parámetros viaja por valor?

Pistas de los propuestos I

- **Propuesto 1:** a es alias de x, así que x sube a 11; y recibe el 12 por *w. Imprime $x = 11$ y $z = 8$: el mismo resultado de la versión con punteros, con la llamada de la versión por valor.
- **Propuesto 2:** los extremos viven en $\&A[0]$ y $\&A[n - 1]$; la función nueva no mueve datos, solo le entrega esas dos direcciones a intercambiar. Queda 5 2 3 4 1.
- **Propuesto 3:** el perímetro se duplica ($14 \rightarrow 28$), pero el área se cuadruplica ($12 \rightarrow 48$): escalar multiplica los dos lados y el área los multiplica entre sí.
- **Propuesto 4:** a viaja por valor y u sigue en 3; *b escribe y v pasa a 6. Imprime 3 6.